DESIGN OF A RESOURCE- AND NETWORK-AWARE CLUSTER SELECTION STRATEGY FOR KARMADA-BASED WORKLOAD PLACEMENT

Vasile CEBAN[1*]

[1] *[Department], group [GROUP], Faculty of Computers, Informatics and Microelectronics, Technical University of Moldova, Chisinau, Republic of Moldova*

*Corresponding author: Vasile Ceban, [EMAIL]

Tutor/coordinator: [Given name FAMILY NAME], [scientific title], [affiliation]

Abstract. *Karmada places workloads on member Kubernetes clusters using cluster health, policy and capacity estimates, but it has no notion of the network latency between clients and clusters. This paper designs a cluster selection strategy that adds such latency to Karmada's existing scheduling process without replacing it. An analysis of the Karmada v1.19.1 source code shows four properties that constrain any extension: score plugins are summed without weights, scores influence selection only under a cluster spread constraint, capacity is estimated from requested rather than used resources, and placement is not revisited when cluster conditions change. Based on these properties, the design separates hard eligibility constraints, evaluated by a filter that reuses Karmada's replica estimators, from a soft score that combines post-placement resource headroom with normalised client-to-cluster round-trip time. Latency measurements are attached to cluster objects by an external probe and carry a validity rule: a cluster without a fresh measurement never benefits from the missing value, and the score falls back to resource-only ranking when no candidate has valid data. A worked example and an analysis of weight sensitivity illustrate the decisions of the design. A validation protocol for a future experimental study is proposed.*

Keywords: *container orchestration, edge-cloud continuum, kubernetes federation, latency, scheduling plugin, telemetry staleness*

1. Introduction

Applications that serve users in several locations are increasingly deployed on more than one Kubernetes cluster. The clusters differ in capacity, in current load and in their network distance to the users. Orchestration across such clusters has been studied as part of the cloud-to-things continuum [1], and several systems coordinate independent clusters, among them the archived KubeFed project [2], Liqo [3] and Karmada [4, 5]. In Karmada, a central control plane receives a workload together with a propagation policy and decides on which member clusters the workload runs. This decision is made by the Karmada scheduler, which, unlike the Kubernetes scheduler, selects clusters rather than nodes.

The information used by this decision is limited. The Karmada scheduler evaluates API availability, cluster affinity, taints and an estimate of how many replicas each cluster can still accept. It does not use any measure of the network path between the clients of a workload and the candidate

clusters. For latency-sensitive services, two clusters with equal free capacity can therefore receive the same preference although one of them is much farther from the clients. Tail latency matters in this setting, because response-time variability at the tail dominates the experience of interactive services at scale [6].

Network-aware scheduling has been studied extensively at node level inside a single cluster [7–11], and recent work has combined resource and latency information for selecting clusters [12–14]. Combining the two kinds of information is therefore not new. What remains open, as discussed in Section 2, is how such a combination fits into the mechanisms Karmada already provides. Existing proposals either delegate the decision to an external optimiser [12] or replace Karmada's cluster scheduler with their own [13]. None of them specifies what happens when the network measurements are missing or out of date.

This paper addresses that question as a design study. It does not report an implementation or measurements; these are planned as a subsequent experimental study. The research questions are:

- RQ1: How can resource and network information be incorporated into Karmada's cluster selection while keeping the existing scheduling behaviour as the foundation?
- RQ2: How should mandatory placement constraints be separated from optimisation objectives?
- RQ3: How can stale or unavailable network measurements be handled so that missing data is never treated as favourable?
- RQ4: Which trade-offs does the resulting design introduce?

The contributions of the paper are the following. First, it analyses the scheduling semantics of Karmada v1.19.1 at the level of the source code and identifies the properties that any custom cluster selection objective has to respect. Second, it proposes a design that keeps the hard eligibility conditions in Karmada's filter stage, reuses Karmada's own replica estimators for the capacity condition, and adds client-to-cluster round-trip time as a soft scoring signal with explicit validity and fallback rules. Third, it illustrates the decisions of the design analytically and proposes a protocol for validating it experimentally.

The paper is organised as follows. Section 2 reviews related work. Section 3 formulates the problem. Section 4 presents the architecture and Section 5 the cluster selection method. Section 6 describes the algorithm and its integration with Karmada. Section 7 analyses the design, Section 8 proposes the experimental validation, Section 9 states the limitations and Section 10 concludes.

2. Background and Related Work

2.1. Scheduling in Kubernetes and in multi-cluster systems

Kubernetes inherited its scheduling model from cluster managers such as Borg, which places tasks on machines using admission control and resource packing [15]. The Kubernetes scheduler selects a node for each pod through a framework of plugins at defined extension points (filtering, scoring, reserving, binding and others), and the scores of the scoring plugins are combined using configurable weights [16]. Surveys classify the large body of work that extends this scheduler by objective, workload type and environment [17–19]. The most recent review covers 2023 to early 2026 and includes scheduling scopes from cloud to edge [20].

Placement across several independent clusters is a separate problem, because each cluster keeps its own control plane and its own node-level scheduler. KubeFed, the earlier federation project of the Kubernetes community, was archived in April 2023 [2]. Liqo shares resources between clusters by representing a remote cluster as a virtual node of the local one [3]. Oakestra, an orchestrator for edge infrastructure that does not build on Kubernetes, uses a hierarchy in which a root component first selects a cluster and the selected cluster then places the task [21]. Karmada follows the same two-level decomposition while remaining Kubernetes-native: a central control plane selects member clusters, and the kube-scheduler of each member cluster selects nodes [5].

2.2. Karmada

A workload submitted to Karmada is an ordinary Kubernetes resource template, for example a Deployment. A PropagationPolicy states the placement rules, such as cluster affinity, spread constraints and the replica scheduling type. The control plane combines template and policy into a ResourceBinding, the Karmada scheduler writes the selected clusters into that binding, and controllers then create the corresponding objects in the member clusters [5]. The scheduler can be extended with filter and score plugins compiled into a custom scheduler binary, and several scheduler instances can run side by side, each serving the workloads whose policy names it [22]. Rebalancing of workloads that are already placed is a separate, explicitly triggered mechanism [23]. An empirical study of Karmada has examined how applications are reconfigured after cluster failures or resource shortage [24]. The parts of the scheduler that matter for this paper are analysed in Section 3.1.

2.3. Network-aware scheduling within one cluster

Santos et al. extended the Kubernetes scheduler to account for the round-trip time between nodes for fog applications [7]. Their later framework, Diktyo, places dependent microservices using an application dependency graph with latency and bandwidth requirements [8]; its plugins are distributed in the Kubernetes scheduler-plugins project [25]. NetMARKS scores nodes using metrics collected by the Istio service mesh [9], and NAS uses latency and bandwidth observed through a Cilium eBPF service mesh [11]. Centofanti et al. argue for measuring latency at the application layer, as perceived by end users, instead of between cluster nodes [10]. Marchese and Tomarchio combine a network-aware scheduler plugin with a descheduler that moves pods when the network state or the traffic between microservices changes [26], and they treat resource load in a separate strategy [27]. Rossi et al. combine elasticity with network-aware placement for nodes that are geographically distributed but managed by one control plane [28].

These works establish that network state is a useful scheduling signal, but they operate on nodes within one control plane. At cluster level the available resource information is aggregated over many nodes, it is refreshed periodically rather than observed per pod, and the relevant network path runs from clients to the entry point of a cluster rather than between nodes.

2.4. Cluster selection with resource and network information

Several works already combine resource and latency information when selecting clusters. Tabatabaei et al. build a service orchestration architecture on Karmada in which a filter plu-

Table 1
Cluster-level placement approaches that use resource and network information

Work	Integration with orchestrator	Signals and decision	Missing-data handling, evaluation
Tabatabaei et al. [12]	Karmada filter plugin calling an external ILP solver	Weighted peak CPU load and latency	Not specified; simulation with fixed RTT values, testbed deployment time
CosmoSpan [13]	Own scheduler; binds through the Karmada API	Multi-criteria score; latency as a filter	Not specified; KWOK simulation, multi-region overhead study
LAIS [14]	Multi-cluster scheduler with latency intents	End-to-end latency (hard or soft intent)	Not specified; comparison with the default Kubernetes scheduler
mck8s [29]	Controllers on KubeFed (archived)	Placement, scaling and bursting policies	Not specified; Grid'5000 testbed
CODECO [30]	Open Cluster Management	CPU, memory, bandwidth, latency, service class	Not specified; preprint without quantitative placement evaluation
This paper	Karmada filter and score plugins; default stages kept	Resource headroom and client-to-cluster RTT; capacity as a hard constraint	Explicit validity and fallback rules; design only, validation planned

gin hands the candidate clusters to an external integer linear programming solver; the objective weighs peak CPU load against latency, and the authors state that the scoring mechanisms of Karmada are not yet included in their flow [12]. Their simulation models latency with fixed round-trip values without running service traffic, and the authors name the focus on initial placement as a limitation. CosmoSpan selects a cluster and a node jointly through a multi-criteria scheduler that binds through the Karmada API, together with deschedulers that react to changes [13]. Its placement score covers CPU, memory, node uptime, bandwidth, latency and fault tolerance; it is evaluated mainly in simulation, and the Karmada scheduler is not part of the comparison because no simulation framework exists for it [13]. LAIS is a multi-cluster scheduler driven by end-to-end latency measurements that either meets a latency constraint or chooses the cluster with the lowest latency, and it is compared with the default Kubernetes scheduler [14]. Earlier, mck8s provided controllers on top of KubeFed that place, scale and burst applications across geo-distributed clusters and was evaluated on the Grid'5000 testbed [29]. The CODECO preprint ranks clusters using CPU, memory, bandwidth, latency and service class over Open Cluster Management [30]. Formal models for placement across federated domains also exist [31, 32], as do learned policies based on deep reinforcement learning for multi-cluster scheduling [33, 34].

Tab. 1 summarises how the closest works relate to the problem of this paper.

Two observations follow from this comparison. First, none of the works defines how a decision is made when network measurements are missing or out of date, although measurements in a real system are periodic and can fail. Second, the works that build on Karmada either delegate the decision to an external solver or bypass the Karmada cluster scheduler, so the interaction of a resource- and network-aware objective with Karmada's own filtering, scoring and selection

Table 2
Properties of the Karmada v1.19.1 scheduler that constrain a custom cluster selection objective

ID	Property	Source file (under pkg/)
P1	Extension points are Filter, Score and an optional NormalizeScore; no per-cycle state, reservation or extender interface exists.	`scheduler/framework/interface.go`
P2	Plugins are compiled into the scheduler binary; the plugin factory receives no arguments.	`scheduler/framework/runtime/registry.go`
P3	Scores of different plugins are summed without weights; a weight map exists but is never filled.	`scheduler/framework/runtime/framework.go`
P4	An error returned by any plugin aborts the scheduling attempt.	`scheduler/framework/runtime/framework.go`
P5	At initial placement both in-tree score plugins return 0.	`scheduler/framework/plugins/clusteraffinity/`, `.../clusterlocality/`
P6	Without a spread constraint every feasible cluster is selected, independent of scores.	`scheduler/core/spreadconstraint/select_clusters.go`
P7	Candidates are ordered by score, then by available replicas, then by name.	`scheduler/core/spreadconstraint/util.go`, `group_clusters.go`
P8	If the top clusters lack capacity, the lowest-scored one is replaced by the remaining cluster with most available replicas, not by the next-best score.	`scheduler/core/spreadconstraint/select_clusters_by_cluster.go`
P9	Available replicas are the minimum over the registered replica estimators; the estimator registry is exported.	`scheduler/core/util.go`, `estimator/client/interface.go`
P10	Cluster resources are reported as Allocatable, Allocated and Allocating, all based on resource requests.	`controllers/status/cluster_status_controller.go`
P11	Placement is not revisited when cluster status changes; only policy, replica or explicit triggers cause rescheduling.	`scheduler/scheduler.go`, `scheduler/event_handler.go`

stages has not been described. This paper addresses both points at the design level. The empirical comparison against the default Karmada scheduler is left to the validation proposed in Section 8.

3. Problem Formulation and Research Objectives

3.1. Scheduling semantics of Karmada relevant to cluster selection

The design is based on an analysis of the Karmada source code at release v1.19.1 (commit c64ba38) [4]. Tab. 2 lists the properties that determine how a custom selection objective can be integrated. Each property is given with the source file in which it can be verified, so that the analysis can be repeated for later releases.

Three consequences follow. Because of P3, weighting between resource and network criteria cannot be configured across separate plugins and has to be computed inside one plugin. Because of P6, scores only influence the outcome when the policy limits the number of selected clusters, for example to one. Because of P5 and P7, the default scheduler, for a new workload restricted to one cluster, selects the cluster with the largest estimated number of available replicas. The default

Table 3
Notation

Symbol	Meaning	Unit / domain
$\mathscr{C}$, $\mathscr{C}_F(w)$	Member clusters; clusters eligible for w	sets
n_w	Number of replicas of w	integer ≥ 1
d_w^k	Request of one replica for resource $k \in \mathscr{K} = \{\text{cpu}, \text{mem}\}$	millicores; bytes
A_c^k, U_c^k, P_c^k	Allocatable, Allocated and Allocating amount of k in cluster c	as d_w^k
$F_c^k = A_c^k - U_c^k - P_c^k$	Free requestable amount of k	as d_w^k
$m_c(w)$	Available replicas of w in c (minimum over estimators)	integer ≥ 0
ℓ_c, t_c, q_c	Client-to-cluster RTT (median), time of newest sample, number of samples	ms; s; integer
ℓ_{ref}	RTT at which network suitability becomes zero	ms
τ, $q_{\min}$	Freshness horizon; minimum number of samples	s; integer
α	Weight of resource suitability ($1 - \alpha$ for network suitability)	$[0, 1]$
$R_c(w)$, N_c, $S_c(w)$	Resource suitability, network suitability, final score	$[0, 1]$, $[0, 1]$, $\{0, \dots, 100\}$

behaviour is therefore already capacity-oriented, and it favours large clusters, since the criterion is an absolute replica count.

3.2. Problem statement

The paper considers the initial placement of a workload w with n_w replicas on exactly one member cluster. Single-cluster placement keeps all replicas, and therefore all communicating components of a single-tier service, in one cluster. Splitting a workload across clusters by score would require changes to Karmada's replica assignment rather than a plugin, and it is not considered here. The notation is summarised in Tab. 3.

The selection problem is to choose

$$c^*(w) = \arg \max_{c \in \mathscr{C}_F(w)} S_c(w), \tag{1}$$

where $\mathscr{C}_F(w)$ contains only clusters that satisfy all mandatory conditions, and $S_c(w)$ expresses preferences among them. Eq. (1) thus separates the two kinds of criteria by construction: a cluster outside $\mathscr{C}_F(w)$ cannot be selected whatever its score. Ties are resolved by the order of property P7.

3.3. Research objectives and design requirements

The research questions of Section 1 translate into the following design requirements.

- D-R1 (integration): use only the documented extension mechanism (P1, P2) and keep the default Karmada stages unchanged, so that the extension can be disabled by selecting the default scheduler.
- D-R2 (separation): mandatory conditions are evaluated as filters and never traded against preferences; preferences are evaluated as scores.
- D-R3 (consistency): the capacity condition must agree with the capacity view used by Karmada's own selection stage, so that the replacement rule P8 does not override the ranking.
- D-R4 (safe telemetry): missing, malformed or stale network measurements must neither cause a

scheduling error (P4) nor improve the score of a cluster.

- D-R5 (analysability): the score must be a closed-form function of observable inputs, so that decisions can be predicted and later tested.

4. Proposed Architecture

Fig. 1 shows the proposed architecture. White components exist in Karmada and are used without modification. Shaded components are proposed in this paper. They are not implemented yet; the implementation is part of the planned experimental study.

4.1. Components

The *extended karmada-scheduler* is the standard Karmada scheduler built with three additional plugins through the documented out-of-tree registration (property P2) [22]. All in-tree plugins remain enabled. *CapacityFit* is a filter that enforces the capacity condition by calling Karmada's replica estimators (P9). *NetResourceScore* is a score plugin that combines resource headroom with network suitability; its normalisation step applies the validity rules for network measurements. *NetLatency* is an optional filter for workloads that require a hard latency bound; it is disabled unless explicitly configured.

The *latency probe* is an external controller placed close to the client population that a scheduler instance represents. It periodically measures the round-trip time to the service entry point of each member cluster, for example by timing a TCP connection set-up, and writes the result to annotations of the corresponding Cluster object in the Karmada control plane. The probe is not on the scheduling path: a failure of the probe changes only the validity of the stored measurements.

Monitoring systems such as Prometheus may observe the scheduler and the probe, but the decision does not query them. Querying an external service inside a score plugin would add network delay to every decision and could fail, and a failure aborts the scheduling attempt (P4).

4.2. Telemetry contract

The probe and the scheduler share the annotations listed in Tab. 4. Annotations are used instead of labels for a specific reason. A change of Cluster labels makes the scheduler re-enqueue the bindings whose cluster affinity can match the changed cluster, whereas a change of annotations triggers no re-enqueueing (P11). With measurements refreshed every few seconds, labels would cause continuous scheduling activity. The scheduler reads the current Cluster objects from its informer cache at the start of each decision, so annotations written by the probe are visible at the next decision without causing additional scheduling activity.

4.3. Existing functionality and proposed extension

Tab. 5 separates what Karmada provides from what the design adds. The extension does not modify Karmada's selection, replica assignment, propagation, rebalancing or failover mechanisms. A workload is handled by the extension only if its PropagationPolicy names the extended scheduler; all other workloads continue to use the default scheduler.

Table 4
Cluster annotations written by the latency probe (prefix is a placeholder)

Annotation	Format	Meaning
`netaware.lab/rtt-ms`	decimal	ℓ_c, median of the last q_c samples, in ms
`netaware.lab/rtt-measured-at`	RFC 3339, UTC	t_c, time of the newest sample
`netaware.lab/rtt-samples`	integer	q_c, number of samples behind ℓ_c
`netaware.lab/rtt-vantage`	string	client population that the value represents

Table 5
Existing Karmada functionality and proposed extensions

Element	Existing	Role in the design
Filter/score framework, plugin registration, scheduler name	yes	used unchanged
Health taints and taint toleration filter	yes	mandatory condition H1
Resource summary (Allocatable, Allocated, Allocating)	yes	input of resource suitability R_c
Replica estimators	yes, in selection	also called before ranking (CapacityFit)
Spread constraint with `maxGroups`, tie-break order	yes	used unchanged
Client-to-cluster latency measurement	no	latency probe, annotations
Resource- and network-aware score with validity rules	no	NetResourceScore
Hard latency bound	no	NetLatency (optional)
Rebalancing, descheduling, failover	yes, separate	not modified

5. Cluster Selection Methodology

5.1. Resource-capacity information available in Karmada

Karmada's cluster status controller, or the Karmada agent for clusters in pull mode, lists the nodes and pods of each member cluster and writes three aggregate quantities to the Cluster object, by default every 10 s and only when they have changed [4]. *Allocatable* A_c^k is the sum of the allocatable amount of resource k over all listed nodes, including nodes that are not ready or are cordoned. *Allocated* U_c^k is the sum of the resource requests of pods that are bound to a node and have not terminated, in all namespaces. *Allocating* P_c^k is the sum of the requests of pods that are not yet bound to a node. The request of a pod is the sum of its container requests, raised to the largest init-container request if that is higher, plus the pod overhead.

All three quantities describe requested, or reserved, resources. None of them reflects measured utilisation. This has three consequences for any decision based on them. Pods without requests contribute nothing, so a cluster loaded by such pods appears emptier than it is. Pending pods that will never fit still count in P_c^k, which is conservative. Nodes that cannot accept pods still count in A_c^k, which is optimistic. The design uses this information anyway, because it is the same request-based view that Kubernetes uses to admit pods to nodes, it is already present in the scheduler's cache, and it does not depend on an external monitoring system.

Aggregate quantities do not show fragmentation. A pod must fit on a single node, so a free amount $F_c^k \geq d_w^k$ does not guarantee that one replica fits. Karmada's replica estimators handle this to different degrees. The general estimator divides the free aggregate amount by the per-replica request when no resource models are available. When resource models are enabled, which is the default, each node is assigned to a grade according to its free resources and represented by the lower bound of that grade, which is conservative. With the default grade boundaries, a node with less than one free CPU or less than 4 GiB of free memory falls into the lowest grade, whose lower bound is zero, and contributes no capacity. Clusters built from small nodes are therefore underestimated unless finer resource models are configured. An optional accurate estimator simulates node-level placement inside each member cluster through a remote call. Disabling the controller option `enable-cluster-resource-modeling` is not a way to switch the models off: it stops the collection of the whole resource summary, after which the estimate becomes zero. The models are controlled by a separate feature gate.

5.2. Mandatory eligibility constraints

A cluster belongs to the eligible set $\mathscr{C}_F(w)$ only if all of the following conditions hold. They are evaluated in the filter stage, before any score is computed.

- H1 (existing): the cluster serves the API of the workload, matches the cluster affinity of the policy, and has no taint that the workload does not tolerate. Unhealthy clusters receive `not-ready` or `unreachable` taints automatically, so health is covered by this condition.
- H2 (capacity): the whole workload fits,

$$m_c(w) = \min_{e \in E} \widehat{m}_c^e(w) \geq n_w, \tag{2}$$

  where E is the set of replica estimators registered in the scheduler and $\widehat{m}_c^e(w)$ is the estimate of estimator e.
- H3 (data presence): the resource summary of the cluster exists. Otherwise $m_c(w) = 0$ and H2 fails.
- H4 (optional latency bound): if a hard bound $\ell_{\max}$ is configured, the measurement of c must be valid in the sense of Section 5.5 and $\ell_c \leq \ell_{\max}$.

The condition in Eq. (2) uses the same estimators that Karmada's selection stage uses, as required by D-R3. Without H2 in the filter stage, a top-scored cluster that lacks capacity would be replaced by the cluster with most available replicas (P8), which discards the ranking. With H2, the clusters that reach the scoring stage are those that the selection stage would also accept. Fragmentation is handled to the extent provided by the configured estimator; the plugin does not re-implement node-level fitting.

H4 is evaluated fail-closed: a cluster without a valid measurement is not eligible. A hard bound is a guarantee, and a guarantee cannot be checked without data. H4 is therefore kept out of the default configuration, in which latency is only a preference.

5.3. Resource suitability

Resource suitability is the fraction of the bottleneck resource that remains free after the workload is placed:

$$R_c(w) = \mathrm{clip}\left(\min_{k \in \mathcal{K}} \frac{F_c^k - n_w\, d_w^k}{A_c^k},\ 0,\ 1 \right). \tag{3}$$

The minimum follows the resource that limits the cluster, so a cluster with free CPU but little free memory does not obtain a high value. Normalising by A_c^k makes the value independent of cluster size. This is the intended difference from the default tie-break (P7), which prefers the cluster with the largest absolute number of available replicas and therefore tends towards large clusters. Aggregate values are acceptable in Eq. (3) because R_c is only a preference; whether the replicas fit on nodes is already ensured by H2.

5.4. Network suitability

For a single-tier service placed on one cluster, the relevant network path runs from the clients to the cluster. Network suitability is therefore based on the client-to-cluster round-trip time ℓ_c, measured from a vantage point that represents the clients:

$$N_c = \mathrm{clip}\left(1 - \frac{\ell_c}{\ell_{\mathrm{ref}}},\ 0,\ 1 \right). \tag{4}$$

The reference value ℓ_{ref} is derived from the latency budget of the service, for example from the share of the response-time objective that may be spent on the network. Latency between services becomes relevant only when the components of an application are split across clusters, which single-cluster placement excludes.

Eq. (4) uses an absolute reference instead of normalising across the current candidates. With min-max normalisation over two or three candidates, the best cluster always receives 1 and the worst 0, whether they differ by 1 ms or by 100 ms. The absolute form keeps scores comparable between decisions and gives α a stable meaning. The median of the last q_c samples is used for ℓ_c because it is not affected by single outliers.

5.5. Validity of network measurements

A measurement of cluster c is *valid* at time t if its annotations are present and well formed, it refers to the vantage point of the scheduler instance, it is based on enough samples and it is recent:

$$\mathrm{valid}_c(t) \iff q_c \geq q_{\min} \ \wedge\ t - t_c \leq \tau. \tag{5}$$

The horizon τ is set to a small multiple of the probe period, for example three periods, so that a single delayed measurement does not invalidate a cluster. Let $\mathcal{V} \subseteq \mathcal{C}_F(w)$ be the eligible clusters with a valid measurement. The final score is

$$S_c(w) = \begin{cases} \mathrm{round}(100\,[\alpha R_c(w) + (1-\alpha) N_c]), & \mathcal{V} \neq \emptyset,\ c \in \mathcal{V}, \\ \mathrm{round}(100\,\alpha R_c(w)), & \mathcal{V} \neq \emptyset,\ c \notin \mathcal{V}, \\ \mathrm{round}(100\,R_c(w)), & \mathcal{V} = \emptyset. \end{cases} \tag{6}$$

The second case sets $N_c = 0$ for a cluster without a valid measurement: missing data is treated as the least favourable network value, never as a neutral or good one (D-R4). The cluster can still be selected if its resource headroom compensates. The third case applies when no candidate

has a valid measurement: the decision falls back to resource-only ranking, so that no cluster is penalised relative to another for a failure that affects all of them. Two alternatives were rejected. A neutral value $N_c = 0.5$ would let an unmeasured cluster outrank a cluster known to be slow. Excluding unmeasured clusters would turn a preference into a mandatory condition and reduce availability when the probe fails. A malformed annotation is treated as missing; it never produces an error, because an error would abort the scheduling attempt (P4).

5.6. Weights and ties

The weight α expresses how much resource headroom is preferred over proximity. It is a parameter of a scheduler instance, not of a workload. The score function receives only the specification of the binding, not its annotations, so per-workload parameters would need additional machinery. Workloads choose between configurations by naming a scheduler instance in their PropagationPolicy. The paper does not prescribe a value of α; Section 7 derives for which ranges of α the decision changes. Scores are integers between 0 and 100, so differences below half a point are lost by rounding. Remaining ties are resolved by Karmada's order (P7), which makes the selection a deterministic function of the cluster snapshot.

6. Algorithm Design and Integration with Karmada

6.1. Position of the plugins in the scheduling pipeline

Karmada processes a binding in the stages shown in Fig. 2: the scheduler takes a snapshot of the Cluster objects, filters the clusters, scores the feasible ones, selects clusters according to the spread constraints, and assigns replicas to the selected clusters [4]. The proposed plugins occupy only the filter, score and normalisation stages. Selection and assignment are Karmada's own. To make scores effective (P6) and to place the whole workload on one cluster, the PropagationPolicy of a workload handled by the extension uses divided replica scheduling with the aggregated preference and a cluster spread constraint with `minGroups` and `maxGroups` equal to one. Duplicated replica scheduling is not used, because in that mode the selection stage ignores available resources.

6.2. Algorithm

Algorithm 1 gives the procedures of the three plugins. Score computes the resource term for each cluster separately, because Karmada calls it once per cluster. The validity rules of Eq. (6) depend on the whole candidate set, which in Karmada is visible only in the normalisation step (P1); the final score is therefore computed there. Every procedure returns a result for malformed input instead of an error.

6.3. Integration and configuration

The plugins are registered in a custom build of the Karmada scheduler through the out-of-tree registration function, and the parameters are passed as command-line options of that binary (P2) [22]. Because the score plugins are summed without weights (P3), the weighting of resource and network suitability is computed inside NetResourceScore and not by combining two plugins. The in-tree plugins remain enabled. At initial placement their scores are zero (P5), so the total score equals $S_c(w)$, and on a later rescheduling the existing locality preference of Karmada continues to

Algorithm 1 Filter, score and normalisation procedures of the proposed plugins

Require: binding with n_w, d_w; parameters α, ℓ_{ref}, τ, $q_{\min}$, vantage v, optional $\ell_{\max}$

procedure CapacityFit(w, c) ▷ filter; H2, H3
  if resource summary of c is absent **then return** Unschedulable
  end if
  $m \leftarrow \min_{e \in E} e.\text{MaxAvailableReplicas}(c, w)$ ▷ Karmada estimators
  if $m < n_w$ **then return** Unschedulable
  end if
  return Success
end procedure
procedure NetLatency(w, c) ▷ optional filter; H4, fail-closed
  if $\ell_{\max}$ not configured **then return** Success
  end if
  if $\neg$ Valid(c, now) **or** $\ell_c > \ell_{\max}$ **then return** Unschedulable
  end if
  return Success
end procedure
procedure Score(w, c) ▷ called once per feasible cluster
  $R \leftarrow \text{clip}\left(\min_k (A_c^k - U_c^k - P_c^k - n_w d_w^k)/A_c^k, 0, 1\right)$ ▷ Eq. (3)
  return round($100R$) ▷ provisional value
end procedure
function Valid(c, t)
  parse ℓ_c, t_c, q_c, v_c from the annotations of c; **if** parsing fails **return false**
  return $v_c = v \wedge q_c \geq q_{\min} \wedge t - t_c \leq \tau$ ▷ Eq. (5)
end function
procedure NormalizeScore(list L of (cluster, score)) ▷ sees all candidates
  $\mathscr{V} \leftarrow \{c \in L : \text{Valid}(c, \text{now})\}$
  for each $c \in L$ **do**
    recompute R_c from the snapshot ▷ avoids rounding twice
    if $\mathscr{V} = \emptyset$ **then** $S_c \leftarrow \text{round}(100R_c)$ ▷ resource-only fallback; counted in a metric
    else
      $N_c \leftarrow \text{clip}(1 - \ell_c/\ell_{\text{ref}}, 0, 1)$ **if** $c \in \mathscr{V}$ **else** 0 ▷ missing is never favourable
      $S_c \leftarrow \text{round}\left(100[\alpha R_c + (1 - \alpha) N_c]\right)$
    end if
  end for
  return Success
end procedure

apply unchanged.

Several scheduler instances can run side by side, each with its own scheduler name and leader-election lock [22]. This allows the same Karmada installation to offer the default behaviour, a resource-only configuration ($\alpha = 1$) and resource- and network-aware configurations ($0 < \alpha < 1$). A workload selects a configuration through the scheduler name in its PropagationPolicy, and removing the name returns it to the default scheduler. The same mechanism provides the comparison setting of the validation plan in Section 8.

Two configuration details affect correctness. First, when resource models are enabled with the default grades, clusters composed of small nodes are underestimated (Section 5.1); such installations should define finer models or enable the accurate estimator. Second, until the pods of a newly placed workload appear in the member cluster, its demand is absent from the resource

Table 6
Illustrative cluster states (chosen inputs, not measurements) and resulting values

Quantity	edge	regional	central
A^{cpu} (m) / A^{mem} (GiB)	4000 / 8	8000 / 16	16000 / 32
F^{cpu} (m) / F^{mem} (GiB)	1200 / 3	4000 / 8	12000 / 24
ℓ_c (ms)	5	25	60
$m_c(w)$ (replicas), H2 satisfied	4, yes	16, yes	48, yes
$R_c(w)$, Eq. (3)	0.050	0.375	0.688
N_c, Eq. (4)	0.95	0.75	0.40
$S_c(w)$ with $\alpha = 0.5$, Eq. (6)	50	56	54

summary. Karmada can account for such in-flight placements through a feature gate that is alpha and disabled by default in v1.19.1. Without it, several placements made within one status interval may select the same cluster. This affects the default scheduler in the same way.

7. Design Analysis and Discussion

This section analyses the decisions that the design produces. The example below uses chosen input values to illustrate the decision rules. The values are not measurements, and the example makes no statement about response time or resource utilisation, which can only be established experimentally (Section 8).

7.1. Illustrative scenario with conflicting criteria

Three single-node member clusters, called edge, regional and central, are described in Tab. 6. A workload with $n_w = 4$ replicas requests 250 millicores of CPU and 512 MiB of memory per replica, i.e. 1000 millicores and 2 GiB in total. The parameters are $\ell_{\text{ref}} = 100$ ms and, for strategy C, $\alpha = 0.5$; all measurements are valid. Because the clusters have a single node, the aggregate summary path of the estimator applies without fragmentation, and $m_c(w) = \min_k \lfloor F_c^k / d_w^k \rfloor$.

The four configurations of the extended scheduler select different clusters. The default scheduler (A) gives every cluster a score of zero (P5) and therefore selects central, which has the largest number of available replicas (P7). The resource-only configuration (B, $\alpha = 1$) also selects central, because central has the largest relative headroom. The combined configuration (C, $\alpha = 0.5$) selects regional. A latency-only configuration ($\alpha = 0$) would select edge, which is closest to the clients but would retain only 5 % of its CPU as free requestable capacity after the placement. In this example, C avoids both the cluster with the least headroom and the most distant cluster. Whether this choice leads to lower response times than A or B depends on how the workload behaves under load at each cluster, which the validation in Section 8 is designed to measure.

7.2. Sensitivity to the weight

For fixed cluster states, every score in Eq. (6) is linear in α. In the example, $S_{\text{edge}}/100 = 0.95 - 0.900\,\alpha$, $S_{\text{regional}}/100 = 0.75 - 0.375\,\alpha$ and $S_{\text{central}}/100 = 0.40 + 0.2875\,\alpha$. Equating the lines gives two switching points: edge is selected for $\alpha < 0.381$, regional for $0.381 < \alpha < 0.528$ and central for $\alpha > 0.528$ (Fig. 3). Integer rounding can move a switching point by up to one score point.

Table 7
Selection in the scenario of Tab. 6 under missing data and a hard bound ($\alpha = 0.5$)

Condition	Scores (edge, regional, central)	Selected
All measurements valid	50, 56, 54	regional
Measurement of regional stale	50, 19, 54	central
All measurements stale (fallback)	5, 38, 69	central
Hard bound $\ell_{max} = 30$ ms (H4)	50, 56, ineligible	regional

Two properties follow. First, for any cluster state the selection changes only at a finite number of values of α, which can be computed in advance; this makes the behaviour of a configuration predictable (D-R5). Second, the width of the intermediate region depends on the cluster states, so a single α is not optimal for all situations. The switching points also give concrete predictions that an experiment can test.

7.3. Behaviour with missing and stale measurements

Tab. 7 applies the validity rules of Section 5.5 to the same scenario. When only the measurement of regional is stale, regional obtains $N = 0$ and is no longer preferred; the stale value neither helps nor removes the cluster. When all measurements are stale, the scheduler behaves exactly like the resource-only configuration. When the optional hard bound H4 is used with $\ell_{max} = 30$ ms, central becomes ineligible and the remaining clusters are ranked as before.

7.4. Complexity and overhead

For $|\mathscr{C}|$ clusters and $|\mathscr{K}|$ resources, the score and normalisation procedures perform $O(|\mathscr{C}||\mathscr{K}|)$ arithmetic operations and read only the in-memory snapshot. The capacity filter calls each registered estimator once per cluster. With the general estimator this is a local computation; the accurate estimator adds a remote call per decision. Karmada already calls the estimators in its selection stage, so the filter repeats an existing computation rather than adding a new kind of cost. The probe costs one measurement and one annotation update per cluster and period, outside the scheduling path. These are analytical statements; the actual overhead has to be measured (Section 8).

7.5. Design decisions and trade-offs

The design keeps Karmada's own stages and adds information instead of replacing the decision procedure. Compared with an external optimiser [12], it gives up global optimality across several workloads in exchange for a closed-form score that is evaluated in the existing pipeline. Compared with a scheduler that replaces Karmada's cluster selection [13], it gives up joint selection of cluster and node in exchange for compatibility with Karmada's policies, estimators and default behaviour.

Reusing Karmada's estimators for the capacity condition makes the filter consistent with the selection stage, but it also inherits the estimators' limitations: an optimistic aggregate estimate on clusters with several nodes, or a conservative estimate with coarse resource models. Using the request-based resource summary avoids a dependency on a monitoring system, at the cost of ignoring unrequested load. Treating missing measurements pessimistically prevents an unmeasured cluster from being preferred, but a cluster whose probe fails is disadvantaged even if it is close to the clients; the fallback limits this effect to the case of partial failure. Annotations avoid scheduling

churn but are not versioned, so the probe and the scheduler have to agree on the schema. Finally, the single client vantage point keeps the model simple but represents only one client population; several populations would require one measurement per population and a way for a workload to name its population.

8. Proposed Experimental Validation for Future Research

The design has not been implemented or measured. This section defines the experimental study that will test it as part of subsequent work.

Questions. The study will answer four questions: (V1) whether the network term changes the response-time distribution relative to the resource-only and default configurations when all clusters have sufficient capacity; (V2) whether, when the cluster closest to the clients has little headroom, the combined score avoids the saturation that latency-only selection causes, and at which latency cost; (V3) whether selection under stale and missing measurements follows Tab. 7; and (V4) how much time the plugins add per decision and how this grows with the number of clusters.

Compared configurations. The study compares the default scheduler (A), the resource-only configuration (B, $\alpha = 1$), the combined configuration (C, $\alpha \in \{0.25, 0.5, 0.75\}$) and a latency-only ablation ($\alpha = 0$). For a fair comparison, all configurations use the same workloads, the same PropagationPolicy (divided, aggregated, one cluster), the same estimator settings and the same clusters, and they differ only in the scheduler name.

Environment. A Karmada control plane manages three member clusters with different allocatable resources, created with a lightweight Kubernetes distribution on local virtual machines. Client-to-cluster delay is emulated with the Linux traffic-control queueing discipline netem, load is generated with k6, and observations are collected with Prometheus. Workloads declare resource requests, which the estimators require, and are CPU-bound with limits, so that saturation affects their response time.

Scenarios. Five scenarios are planned: heterogeneous capacity without pressure; CPU and memory pressure on one cluster; latency degradation on one cluster; the conflict of Section 7.1, with the closest cluster having the least headroom; and stale or missing measurements, produced by stopping the probe for one cluster and then for all clusters.

Metrics. Response time at the 50th, 95th and 99th percentiles and the rate of responses exceeding a response-time objective [6]; the selected cluster per run; the share of requested resources per cluster; measured CPU and memory utilisation for observation; and decision time from the scheduler's duration metrics for plugins and complete scheduling attempts [4], complemented by micro-benchmarks for 3 to 100 synthetic clusters, since the shortest histogram bucket of these metrics is 1 ms.

Protocol and analysis. Each combination of scenario and configuration will be repeated at least ten times in randomised order, with a warm-up period, a fixed measurement window and a reset of the environment between runs. Percentiles will be computed per run and summarised by their median over runs with bootstrap confidence intervals; differences will be tested with non-parametric tests and reported with effect sizes, including results that show no difference. The reporting will follow the methodological principles for reproducible performance evaluation in

cloud computing [35].

Threat to validity. With emulated delay, part of any latency reduction exists by construction, because the scheduler avoids the clusters to which delay was added. The analysis will therefore relate observed differences to the emulated ones and concentrate on the conflict scenario, in which proximity and capacity pull in opposite directions.

9. Limitations

The paper presents a design, not an evaluated system. No claim is made that the design improves response time or resource utilisation; such claims require the study of Section 8.

The analysis of Karmada refers to release v1.19.1. Several of the properties in Tab. 2, such as the unweighted sum of scores and the replacement rule in the selection stage, are internal behaviour rather than documented interfaces and may change in later releases. The analysis has to be repeated for each release on which the design is implemented.

The resource term uses requested resources and aggregate values. It does not see load from pods without requests, and its accuracy for the capacity condition is limited by the configured estimator. The network term uses one client vantage point and the round-trip time to the cluster entry point; it does not cover several client populations, bandwidth, or latency between services of a workload split across clusters, which single-cluster placement excludes by assumption. The freshness check compares timestamps written by the probe with the scheduler's clock and assumes synchronised clocks.

The design concerns initial placement only. Because Karmada does not revisit placements when cluster conditions change (P11), a workload stays on the selected cluster even if the conditions that led to the decision no longer hold.

10. Conclusions and Future Work

The paper designed a cluster selection strategy that adds client-to-cluster latency to Karmada's scheduling without replacing Karmada's own stages. Regarding RQ1, the analysis of the Karmada v1.19.1 source code showed that a custom objective has to be computed inside a single score plugin, that it only takes effect together with a cluster spread constraint, and that Karmada's default behaviour for a new single-cluster workload already prefers the cluster with the most available replicas. Regarding RQ2, mandatory conditions are evaluated in the filter stage, and the capacity condition reuses Karmada's replica estimators so that it agrees with the selection stage. Regarding RQ3, a network measurement contributes to the score only if it is recent and based on enough samples; a missing measurement is treated as the least favourable value, and the score falls back to resource-only ranking when no candidate has a valid measurement. Regarding RQ4, the design trades global optimality and joint node selection for compatibility with Karmada and a closed-form score whose decisions can be predicted, and it inherits the limitations of request-based capacity information.

Future work consists of implementing the plugins and the latency probe, and carrying out the experimental validation of Section 8 against the default and resource-only configurations. Further extensions are the rebalancing of workloads when conditions change, which Karmada supports

only through explicit triggers [23]; the use of measured utilisation alongside requested resources; several client populations; latency between services for workloads split across clusters; and an evaluation of performance, scalability and reliability on larger installations.

References

[1] ULLAH, A.; KISS, T.; KOVÁCS, J., et al. Orchestration in the Cloud-to-Things compute continuum: taxonomy, survey and future directions. *Journal of Cloud Computing* **2023**, *12*(1), 135. https://doi.org/10.1186/s13677-023-00516-5.

[2] KubeFed: Kubernetes Cluster Federation (repository archived on 25 April 2023). accessed on 10 November 2026. Available online: https://github.com/kubernetes-retired/kubefed.

[3] IORIO, M.; RISSO, F.; PALESANDRO, A.; CAMICIOTTI, L.; MANZALINI, A. Computing Without Borders: The Way Towards Liquid Computing. *IEEE Transactions on Cloud Computing* **2023**, *11*(3), pp. 2820–2838. https://doi.org/10.1109/TCC.2022.3229163.

[4] Karmada: Open, Multi-Cloud, Multi-Cluster Kubernetes Orchestration. Source code, release v1.19.1 (commit c64ba38). accessed on 10 November 2026. Available online: https://github.com/karmada-io/karmada/tree/v1.19.1.

[5] Karmada Documentation v1.19: Architecture. accessed on 10 November 2026. Available online: https://karmada.io/docs/core-concepts/architecture.

[6] DEAN, J.; BARROSO, L. A. The tail at scale. *Communications of the ACM* **2013**, *56*(2), pp. 74–80. https://doi.org/10.1145/2408776.2408794.

[7] SANTOS, J.; WAUTERS, T.; VOLCKAERT, B.; DE TURCK, F. Towards Network-Aware Resource Provisioning in Kubernetes for Fog Computing Applications. In *Proceedings of the 2019 IEEE Conference on Network Softwarization (NetSoft)*. 2019, pp. 351–359. https://doi.org/10.1109/NETSOFT.2019.8806671.

[8] SANTOS, J.; WANG, C.; WAUTERS, T.; DE TURCK, F. Diktyo: Network-Aware Scheduling in Container-Based Clouds. *IEEE Transactions on Network and Service Management* **2023**, *20*(4), pp. 4461–4477. https://doi.org/10.1109/TNSM.2023.3271415.

[9] WOJCIECHOWSKI, L.; OPASIAK, K.; LATUSEK, J., et al. NetMARKS: Network Metrics-AwaRe Kubernetes Scheduler Powered by Service Mesh. In *Proceedings of the IEEE INFOCOM 2021 – IEEE Conference on Computer Communications*. 2021, pp. 1–9. https://doi.org/10.1109/INFOCOM42981.2021.9488670.

[10] CENTOFANTI, C.; TIBERTI, W.; MAROTTA, A.; GRAZIOSI, F.; CASSIOLI, D. Latency-Aware Kubernetes Scheduling for Microservices Orchestration at the Edge. In *Proceedings of the 2023 IEEE 9th International Conference on Network Softwarization (NetSoft)*. 2023, pp. 426–431. https://doi.org/10.1109/NETSOFT57336.2023.10175431.

[11] KARUNAI KIRI RAGAVAN, V.; NADIG, D. NAS: A Novel Network-Aware Kubernetes Scheduling Framework Using eBPF Service Mesh. In *Proceedings of the ICC 2025 – IEEE International Conference on Communications*. 2025, pp. 1512–1517. https://doi.org/10.1109/ICC52391.2025.11161235.

[12] TABATABAEI, F.; MANGUES-BAFALLUY, J.; KHALILI, H.; KAHVAZADEH, S.; REQUENA-ESTESO, M. Optimizing CNF Placement for Latency and Resource Efficiency in Multi-Cluster Cloud–Edge Federations. *IEEE Open Journal of the Communications Society* **2025**, *6*, pp. 8376–8388. https://doi.org/10.1109/OJCOMS.2025.3614745.

[13] BREGAR, V.; JURIC, M. B. Toward adaptive workload scheduling in Kubernetes across the edge-cloud continuum. *Cluster Computing* **2026**, *29*(15), 850. https://doi.org/10.1007/s10586-026-06669-7.

[14] CHIARO, C.; MONACO, D.; SACCO, A.; CASETTI, C.; MARCHETTO, G. Latency-aware Scheduling in the Cloud-Edge Continuum. In *Proceedings of the NOMS 2024 – 2024 IEEE Network Operations and Management Symposium*. 2024, pp. 1–5. https://doi.org/10.1109/NOMS59830.2024.10575183.

[15] VERMA, A.; PEDROSA, L.; KORUPOLU, M.; OPPENHEIMER, D.; TUNE, E.; WILKES, J. Large-scale cluster management at Google with Borg. In *Proceedings of the Tenth European Conference on Computer Systems (EuroSys '15)*. 2015, pp. 1–17. https://doi.org/10.1145/2741948.2741964.

[16] Kubernetes Documentation: Scheduling Framework. accessed on 10 November 2026. Available online: https://kubernetes.io/docs/concepts/scheduling-eviction/scheduling-framework/.

[17] CARRIÓN, C. Kubernetes Scheduling: Taxonomy, Ongoing Issues and Challenges. *ACM Computing Surveys* **2022**, *55*(7), pp. 1–37. https://doi.org/10.1145/3539606.

[18] REJIBA, Z.; CHAMANARA, J. Custom Scheduling in Kubernetes: A Survey on Common Problems and Solution Approaches. *ACM Computing Surveys* **2022**, *55*(7), pp. 1–37. https://doi.org/10.1145/3544788.

[19] SENJAB, K.; ABBAS, S.; AHMED, N.; KHAN, A. u. R. A survey of Kubernetes scheduling algorithms. *Journal of Cloud Computing* **2023**, *12*(1), 87. https://doi.org/10.1186/s13677-023-00471-1.

[20] ALHAKIMI, M.; LATIP, R. Enhancing the Kubernetes Scheduler: A State-of-the-Art Review from Cloud to Edge. *Computers* **2026**, *15*(7), 458. https://doi.org/10.3390/computers15070458.

[21] BARTOLOMEO, G.; YOSOFIE, M.; BÄURLE, S.; HALUSZCZYNSKI, O.; MOHAN, N.; OTT, J. Oakestra: A Lightweight Hierarchical Orchestration Framework for Edge Computing. In *Proceedings of the 2023 USENIX Annual Technical Conference (USENIX ATC 23)*. Boston, MA, USA: USENIX Association, 2023, pp. 215–231. Available online: https://www.usenix.org/conference/atc23/presentation/bartolomeo.

[22] Karmada Documentation v1.19: Customize the Scheduler. accessed on 10 November 2026. Available online: https://karmada.io/docs/developers/customize-karmada-scheduler.

[23] Karmada Documentation v1.19: Workload Rebalancer. accessed on 10 November 2026. Available online: https://karmada.io/docs/userguide/scheduling/workload-rebalancer.

[24] PATIÑO-MARTÍNEZ, M.; AZQUETA-ALZÚAZ, A.; AGIRRE, A. Reconfiguration of Applications in Cloud-Edge Multi-Clusters. In *Proceedings of the 2025 IEEE 45th International Conference on Distributed Computing Systems Workshops (ICDCSW)*. 2025, pp. 321–326. https://doi.org/10.1109/ICDCSW63273.2025.00060.

[25] Kubernetes SIG Scheduling: scheduler-plugins, Network-Aware Scheduling. accessed on 10 November 2026. Available online: https://github.com/kubernetes-sigs/scheduler-plugins/tree/master/pkg/networkaware.

[26] MARCHESE, A.; TOMARCHIO, O. Network-Aware Container Placement in Cloud-Edge Kubernetes Clusters. In *Proceedings of the 2022 22nd IEEE International Symposium on Cluster, Cloud and Internet Computing (CCGrid)*. 2022, pp. 859–865. https://doi.org/10.1109/CCGRID54584.2022.00102.

[27] MARCHESE, A.; TOMARCHIO, O. Enhancing the Kubernetes Platform with a Load-Aware Orchestration Strategy. *SN Computer Science* **2025**, *6*(3), 224. https://doi.org/10.1007/s42979-025-03712-z.

[28] ROSSI, F.; CARDELLINI, V.; LO PRESTI, F.; NARDELLI, M. Geo-distributed efficient deployment of containers with Kubernetes. *Computer Communications* **2020**, *159*, pp. 161–174. https://doi.org/10.1016/j.comcom.2020.04.061.

[29] TAMIRU, M. A.; PIERRE, G.; TORDSSON, J.; ELMROTH, E. mck8s: An orchestration platform for geo-distributed multi-cluster environments. In *Proceedings of the 2021 International Conference on Computer Communications and Networks (ICCCN)*. 2021, pp. 1–10. https://doi.org/10.1109/ICCCN52240.2021.9522318.

[30] SOFIA, R. C.; SALOMON, J.; CARROL, R., et al. Towards Scalable Federated Container Orchestration: The CODECO Approach. Preprint, not peer-reviewed. 2026. arXiv: 2601.13351.

[31] FATICANTI, F.; SAVI, M.; DE PELLEGRINI, F.; SIRACUSA, D. Locality-aware deployment of application microservices for multi-domain fog computing. *Computer Communications* **2023**, *203*, pp. 180–191. https://doi.org/10.1016/j.comcom.2023.02.012.

[32] KHAN, S.; KHAN, S. Latency aware graph-based microservice placement in the edge-cloud continuum. *Cluster Computing* **2025**, *28*(2), 88. https://doi.org/10.1007/s10586-024-04824-6.

[33] HUANG, C.; TAN, W.; OU, M.; YANG, E.; LI, Y. KMPS: A Reinforcement Learning Scheduler for Kubernetes Edge-Cloud Systems. *IEEE Internet of Things Journal* **2026**, *13*(8), pp. 16021–16034. https://doi.org/10.1109/JIOT.2026.3658608.

[34] JIANG, J.; LI, Q.; WANG, P.; LIU, Y. DRKC: Deep Reinforcement Learning Enhanced Microservice Scheduling on Kubernetes Clusters in Cloud-Edge Environment. *IEEE Transactions on Cloud Computing* **2025**, *13*(4), pp. 1472–1486. https://doi.org/10.1109/TCC.2025.3624031.

[35] PAPADOPOULOS, A. V.; VERSLUIS, L.; BAUER, A., et al. Methodological Principles for Reproducible Performance Evaluation in Cloud Computing. *IEEE Transactions on Software Engineering* **2021**, *47*(8), pp. 1528–1543. https://doi.org/10.1109/TSE.2019.2927908.

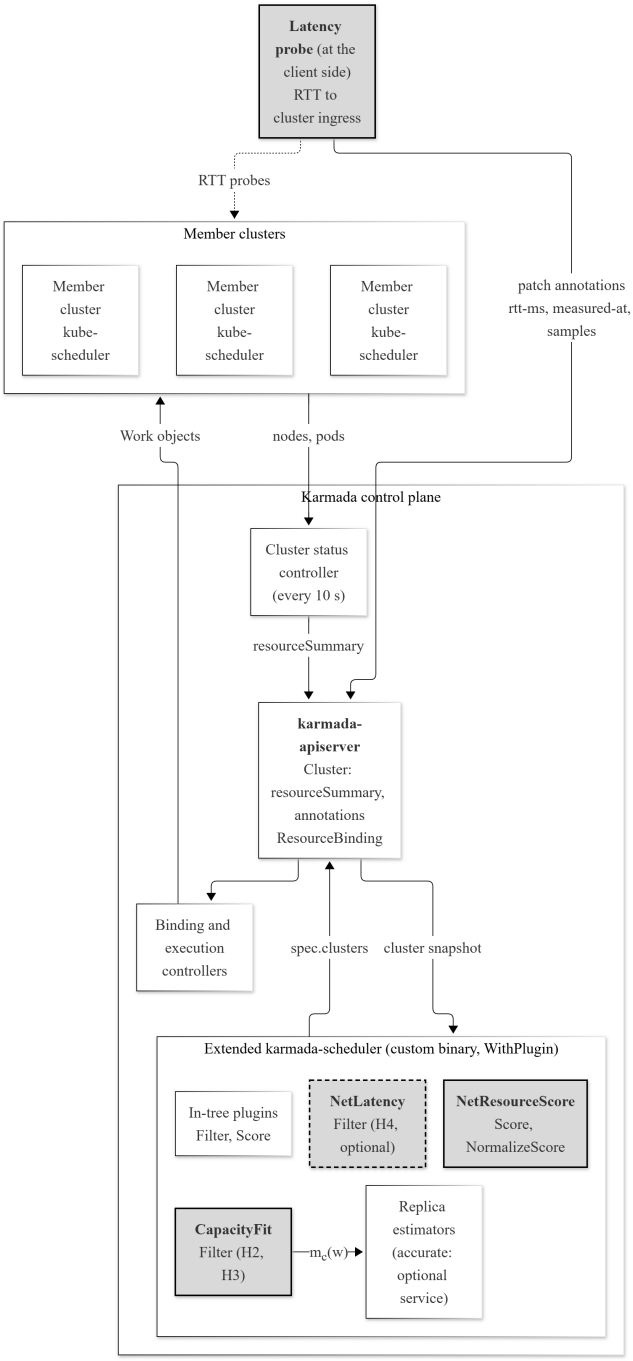


Figure 1. Proposed architecture; shaded boxes are proposed components, white boxes are existing Karmada components

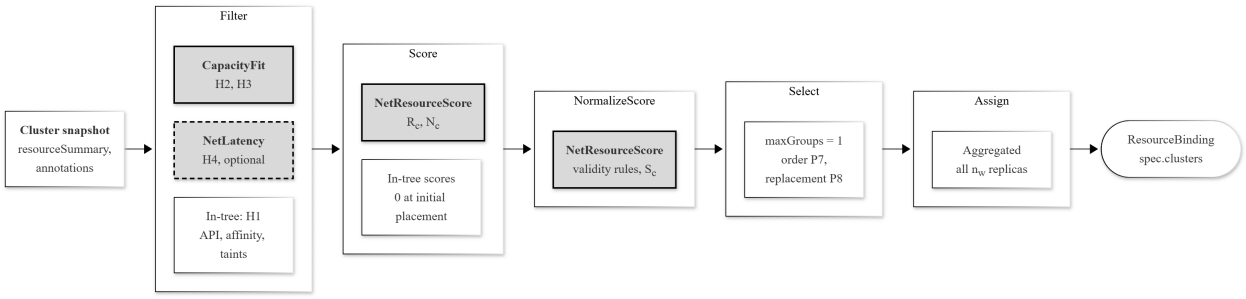


Figure 2. Karmada scheduling stages and the position of the proposed plugins (shaded)

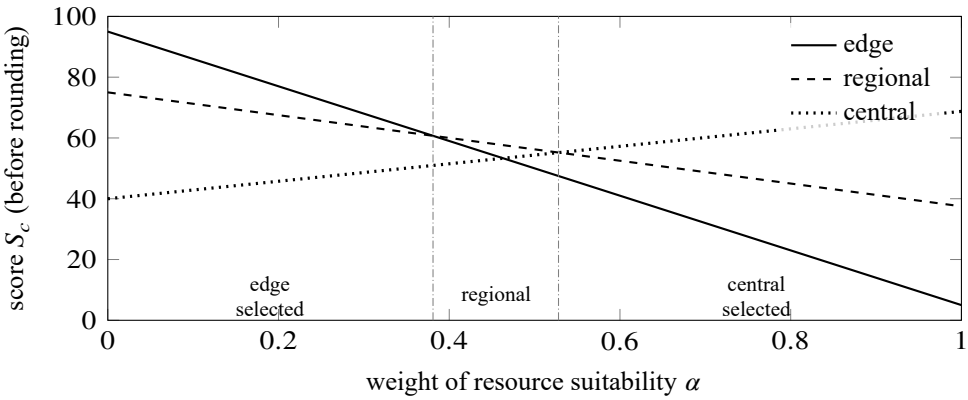


Figure 3. Score of each cluster in the scenario of Tab. 6 as a function of α (analytic, no measured data)